

Week 2- Session 2 - Lab Practical

Relational and NoSQL Database Fundamentals

Section A: Scenario-Based Questions

For each scenario, decide Relational or NoSQL (and which NoSQL type, if applicable), then justify your choice in 2-3 sentences using the concepts from the session (ACID/BASE, schema flexibility, query pattern).

A1. Payment Processing System

A fintech app needs to debit one account and credit another atomically, with strict rollback if either step fails.

ANS: Relational database. ACID confirms atomicity. Relational databases are good for transactions

A2. E-commerce Product Catalog

Products range from books to electronics to furniture — each category has wildly different attributes (e.g., "page count" vs "battery life" vs "dimensions").

ANS: NoSQL. This is a documents database case. The product range requires a database with no enforced schema.

A3. Social Media Friend Recommendations

The app needs to suggest "people you may know" by finding friends-of-friends up to 3 levels deep.

ANS: NoSQL. Particularly GraphDB. The relationship between entities (friends-of-friends) requires such a solution

A4. IoT Sensor Network

Thousands of sensors report temperature readings every second; the system must handle very high write throughput, and reads are mostly "give me the last 24 hours for sensor X."

ANS: NoSQL. For this case, columnar storage would be an advantage. For thousands of sensors, it would be easy to select sensor X without loading the entire data in memory.

A5. Session/Cart Management

A retail site needs to store each logged-in user's shopping cart for fast retrieval by session ID, with no complex querying required.

ANS: NoSQL. Fast retrieval calls of key-value databases, suitable for this case

A6. Financial Reporting Dashboard

A finance team needs to join customer, order, and payment tables to produce monthly revenue reports with strict accuracy.

ANS: Relational. The complex joins, is a proper use case for OLAP relational schemas.

Section B: Multiple Choice Questions

B1. What does a foreign key primarily enforce?

- a) Query performance
- b) Referential integrity between tables
- c) Schema flexibility
- d) Data compression

ANS: B

B2. Which statement about NoSQL is most accurate?

- a) NoSQL means no schema at all
- b) NoSQL databases can never scale
- c) NoSQL favors flexible schema over strict enforcement
- d) NoSQL databases don't support any data validation

ANS: C

B3. In the ACID model, "Isolation" refers to:

- a) Data being stored on isolated servers
- b) Concurrent transactions not interfering with each other
- c) Backups being isolated from production
- d) Tables being isolated from each other

ANS: B

B4. Which NoSQL type is best suited for "friends of friends" traversal queries?

- a) Key-value
- b) Document
- c) Column-family
- d) Graph

ANS: D

B5. BASE stands for:

- a) Basic Availability, Stateless Execution
- b) Basically Available, Soft state, Eventual consistency
- c) Balanced Access, Secure Encryption
- d) Batch Aggregation, Synchronous Execution

ANS: A

B6. A primary key automatically implies:

- a) The column is encrypted
- b) An index is often created for uniqueness enforcement
- c) The table has no foreign keys
- d) The column must be a string type

ANS: B

B7. Which use case is the best fit for a column-family database?

- a) Bank account transfers
- b) Per-user daily time-series metrics with many optional fields
- c) Storing a single user's session token

d) Modeling friendships between users

ANS: B

B8. What is a key trade-off when choosing BASE over ACID?

- a) You gain strong consistency but lose availability
- b) You gain availability/partition tolerance but accept eventual consistency
- c) You gain faster joins but lose indexing
- d) You gain schema enforcement but lose flexibility

ANS: B

B9. Embedding line items inside an order document (vs. referencing them) is a pattern typical of:

- a) Column-family databases
- b) Graph databases
- c) Document databases
- d) Key-value databases

ANS: B

B10. Which scenario is the weakest fit for a graph database?

- a) Fraud ring detection across connected accounts
- b) Mutual friend discovery
- c) Large-scale tabular financial reporting with millions of rows
- d) Recommendation engines based on relationships

ANS: B

Section C: Theory Questions

C1.

Explain the difference between a primary key and a foreign key, and describe one real-world consequence of missing a foreign key constraint.

ANS:

A primary key enforces uniqueness across rows, while a foreign key references a unique key (such as a primary key) in another table, managing relationships between tables. Missing a foreign key constraint can make joins between tables error-prone.

C2.

Compare ACID and BASE across consistency, availability, and typical use cases. Give one example use case for each that was not used in the session notes.

ANS:

- 1. Consistency: Consistency is a more stringent requirement for ACID properties. Here write and schema consistency is required more than in BASE cases.**
- 2. Availability: BASE solutions are needed more when availability is required. Write especially have to be ensured regardless of the schema.**
- 3. Use case:**
 - **ACID: A sports betting platform that collects user bets and ensures wallets and bets are settled properly.**
 - **BASE: For live events on the sports betting platform, high availability and low latency are needed**

C3.

"NoSQL means no schema." Explain why this is a common misconception and describe what actually happens to schema in a NoSQL system.

ANS:

NoSQL does not necessarily mean "no schema"; it means the schema is flexible. This means that the database layer does not enforce schema; that is the application layer's task.

C4.

Describe the four main NoSQL types (document, key-value, column-family, graph) in one sentence each, and give one data engineering example per type.

ANS:

- **Document: data with a flexible schema. Raw information generally, regardless of schema. Eg: storing odds information from different bookmakers with different JSON schemas.**
- **Key-value: Data storage for quick identity-based lookups. E.g., a sports platform requiring easy query into the activities of a particular user**
- **Column-family: Data with many features (columns) where quick aggregation on a single feature may be required. E.g., sports player statistics across different metrics, where it might be necessary to compare a player's defensive statistics over a long period.**
- **Graph: Data with multi-layered relationships between entities (nodes). E.g., analysing teams and opponents across multiple competitions.**

C5.

A team wants to migrate a fixed-schema product table to a flexible-schema document model because product attributes vary heavily by category. What is gained, and what risk should the team watch for after migration?

ANS:

This migration would reduce complexity by eliminating overbloated schemas for entities with always-changing properties. In this case, writing would be easier. However, on read, application layers would need to handle flexible schemas; if not handled properly (validation-on-write), application-layer errors might occur.

Section D: Practical Questions (Try it yourself)

D1. Relational -Build and Query

Using sqlite3, create two tables: employees(employee_id PK, name, dept_id FK) and departments(dept_id PK, dept_name). Insert at least 2 departments and 3 employees. Write a query that joins both tables to list each employee's name alongside their department name.

```
import sqlite3
import pandas as pd
conn = sqlite3.connect("session_2.db")
cur = conn.cursor()

cur.executescript("""
DROP TABLE IF EXISTS employees;
DROP TABLE IF EXISTS departments;
CREATE TABLE departments (
    dept_id INTEGER PRIMARY KEY,
    dept_name TEXT NOT NULL
);

CREATE TABLE employees (
    employee_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    dept_id INTEGER,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);

INSERT INTO departments (dept_id, dept_name) VALUES
    (1, 'Engineering'),
    (2, 'Marketing');

INSERT INTO employees (employee_id, name, dept_id) VALUES
    (1, 'Alice', 1),
    (2, 'Bob', 1),
    (3, 'Charlie', 2);
""")

query = """
SELECT e.name AS employee_name, d.dept_name AS department_name
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id;
"""
df = pd.read_sql(query, conn)
```

D2. ACID - Transaction Rollback

Using sqlite3, create a wallet(user, balance) table. Simulate a transfer of 30 units from one user to another inside a transaction. Deliberately raise an exception before committing, then roll back and print the table to confirm balances are unchanged.

```
import sqlite3

conn = sqlite3.connect("wallet.db")
cur = conn.cursor()

cur.executescript("""
DROP TABLE IF EXISTS wallet;
```

```

CREATE TABLE wallet (
    user TEXT PRIMARY KEY,
    balance INTEGER NOT NULL
);

INSERT INTO wallet (user, balance) VALUES
    ('Alice', 100),
    ('Bob', 50);
"""
conn.commit()

def print_wallet(label):
    print(f"--- {label} ---")
    cur.execute("SELECT * FROM wallet")
    for row in cur.fetchall():
        print(row)

print_wallet("Before transfer attempt")

try:
    cur.execute("BEGIN")
    # Deduct 30 from Alice
    cur.execute("UPDATE wallet SET balance = balance - 30 WHERE user =
'Alice'")
    # Add 30 to Bob
    cur.execute("UPDATE wallet SET balance = balance + 30 WHERE user = 'Bob'")
    # Deliberately raise error before committing
    raise RuntimeError("Simulated failure before commit")
    conn.commit() # never reached

except Exception as e:
    print(f"Transaction failed: {e}")
    conn.rollback()

print_wallet("After rollback")
conn.close()

```

D3. Document Model - Variable Attributes

Using Python dictionaries, create a small in-memory "collection" (list of dicts) of at least 3 products from different categories (e.g., book, laptop, chair), where each product has different fields beyond a shared sku and name. Write a function that filters and returns only products that have a battery_life field.

```

products = [
    {
        "sku": "B001",
        "name": "Wireless Mouse",
        "category": "electronics",
        "battery_life": "12 months",
        "connectivity": "Bluetooth"
    },
    {
        "sku": "B002",
        "name": "Clean Code",
        "category": "book",
        "author": "Robert C. Martin",

```

```

        "pages": 464
    },
    {
        "sku": "B003",
        "name": "UltraBook Pro 14",
        "category": "laptop",
        "battery_life": "18 hours",
        "ram_gb": 16,
        "storage_gb": 512
    },
    {
        "sku": "B004",
        "name": "Ergo Office Chair",
        "category": "chair",
        "material": "mesh",
        "max_weight_kg": 120
    }
]

def filter_by_battery_life(product_list):
    return [p for p in product_list if "battery_life" in p]

products_with_battery = filter_by_battery_life(products)
print(products_with_battery)

```

D4. Key-Value Store - Simulated Cache

Using a Python dict as a key-value store, write code to store 3 user sessions keyed by "session:<id>", then write a function `get_session(session_id)` that returns the session or None if not found (simulating a cache miss).

```

sessions = {
    "session:101": {"user_id": 1, "username": "chef_alice", "role": "admin"},
    "session:102": {"user_id": 2, "username": "dr_bob", "role": "analyst"},
    "session:103": {"user_id": 3, "username": "eng_charlie", "role":
"developer"}
}

def get_session(session_id):
    return sessions.get(f"session:{session_id}", None)

# Example usage
print(get_session(102))
print(get_session(103))
print(get_session(104))

```

D5. Eventual Consistency

Using two Python dicts (`replicaA`, `replicaB`) to simulate replicas, write a value to `replicaA` only, show that reading it from `replicaB` returns nothing (stale/missing), then simulate a sync and show `replicaB` now has the value. Add a short comment explaining which BASE property this demonstrates.

```

# Simulating two database replicas as Python dicts
replicaA = {}
replicaB = {}

```



```

# Write a value to replicaA only
replicaA["user:1"] = {"name": "Alice", "status": "active"}

# Write a value to replicaB only
replicaB["user:2"] = {"name": "Mad Hatter", "status": "inactive"}

print("Read from replicaA (user1):", replicaA.get("user:1"))
print("Read from replicaB (user1):", replicaB.get("user:1")) # None - replica
hasn't synced yet
print("Read from replicaA (user2):", replicaA.get("user:2"))

# This gap (replicaB missing data that replicaA already has) demonstrates
# "Eventual Consistency" from BASE (Basically Available, Soft state, Eventually
consistent):
# writes don't propagate to all replicas instantly, so reads on unsynced
# replicas can be stale or missing, but will converge given enough time.

def sync_replicas(source, target):
    target.update(source)

# Simulate the sync/replication process (A to B)
sync_replicas(replicaA, replicaB)

print("\nAfter sync: A (source) -> B (target)")
print("Read from replicaA:", replicaA.get("user:1"))
print("Read from replicaB:", replicaB.get("user:1")) # Now present -
consistency achieved
print("Read from replicaA:", replicaA.get("user:2"))

# Simulate the sync/replication process (B to A)
sync_replicas(replicaB, replicaA)
print("\nAfter sync: B (source) -> A (target)")
print("Read from replicaA:", replicaA.get("user:2"))
print("Read from replicaB:", replicaB.get("user:1"))

```

D6. Graph - Friends of Friends

Using an adjacency list (dict of lists) representing a small social graph with at least 4 users, write code to compute the friends-of-friends for one user, excluding the user themselves and their direct friends.

```

graph = {
    "Alice": ["Mad Hatter", "Willie Wonka"],
    "Mad Hatter": ["Alice", "Red Queen"],
    "Willie Wonka": ["Alice", "Red Queen", "Chershire Cat"],
    "Red Queen": ["Mad Hatter", "Willie Wonka"],
    "Chershire Cat": ["Willie Wonka"]
}

def friends_of_friends(graph, user):
    if user not in graph:
        print("Warning: User not found")
        return []

    direct_friends = set(graph.get(user, []))
    fof = set()

```

```
for friend in direct_friends:
    for candidate in graph.get(friend, []):
        fof.add(candidate)

# Exclude the user themselves and their direct friends
fof.discard(user)
# fof -= direct_friends

fof = list(fof)
return fof

result = friends_of_friends(graph, "Alicae")
print(result)  # {'Red Queen', 'Cheshire Cat'}
```